

Intro to DevOps — Module 2

Practice Questions — Model Answers (B2 English)

LO4 — Accelerated delivery of multilayer applications using containers

Q1: Create a Deployment named 'web' running nginx:1.25 with 3 replicas using a single imperative kubectl create deployment command, then export its generated YAML to a file.

Use the following imperative command to create the deployment:

```
kubectl create deployment web --image=nginx:1.25 --replicas=3
```

Then export the generated YAML manifest to a file:

```
kubectl get deployment web -o yaml > web-deployment.yaml
```

The `--image` flag specifies the container image, and `--replicas` sets the desired number of pod instances. Exporting with `-o yaml` gives you the full server-side manifest, including fields like `resourceVersion` that `kubectl` added automatically.

Q2: Enable pulling images from DockerHub as an authenticated user to lower the chance of hitting the pull limit. What kind of resource do you need to create?

You need to create a Secret of type `docker-registry` (also called an image-pull secret):

```
kubectl create secret docker-registry dockerhub-creds \ --docker-username=<USER> \ --docker-password=<PASSWORD> \ --docker-email=<EMAIL>
```

Then reference it in the pod spec or the default ServiceAccount:

```
imagePullSecrets: - name: dockerhub-creds
```

DockerHub enforces anonymous pull rate limits (100 pulls/6 hours per IP for unauthenticated users). Authenticating raises the limit and associates pulls with your account.

Q3: Scale 'web' from 3 to 5 replicas two different ways — once with kubectl scale, once by editing the manifest — and show the ReplicaSet that results.

Method 1 — imperative scale command:

```
kubectl scale deployment web --replicas=5
```

Method 2 — edit the manifest in place:

```
kubectl edit deployment web # change spec.replicas from 3 to 5, save and exit
```

Verify the resulting ReplicaSet:

```
kubectl get replicaset -l app=web
```

You will see one ReplicaSet with `DESIRED=5`, `CURRENT=5`, `READY=5`. When you scale a Deployment, it updates the ReplicaSet's desired count; no new ReplicaSet is created because the pod template has not changed.

Q4: Perform a rolling update of 'web' from nginx:1.25 to nginx:1.27 and watch it with kubectl rollout status.

Trigger the update:

```
kubectl set image deployment/web nginx=nginx:1.27
```

Watch the rollout in real time:

```
kubectl rollout status deployment/web
```

Kubernetes replaces pods gradually according to the RollingUpdate strategy. New pods with nginx:1.27 are created while old pods are terminated, ensuring zero downtime. The rollout status command blocks until all replicas are updated or a failure occurs.

Q5: View a Deployment's rollout history and roll back to the previous revision. Explain what the CHANGE-CAUSE column shows and how to populate it.

View rollout history:

```
kubectl rollout history deployment/web
```

Roll back to the previous revision:

```
kubectl rollout undo deployment/web
```

The CHANGE-CAUSE column shows an annotation explaining why the change was made. It is empty by default. You populate it by adding the kubernetes.io/change-cause annotation at the time of the update:

```
kubectl annotate deployment web kubernetes.io/change-cause="Updated image to nginx:1.27"
```

Alternatively, use --record (deprecated) or set the annotation in the manifest.

Q6: Set the update strategy to RollingUpdate with maxSurge: 1 and maxUnavailable: 0, and explain the effect on availability during an update.

Add or edit the strategy block in the Deployment manifest:

```
strategy:   type: RollingUpdate   rollingUpdate:   maxSurge: 1
maxUnavailable: 0
```

Effect on availability: with maxUnavailable: 0, no existing pod is terminated until a new pod is fully Running and Ready. With maxSurge: 1, at most one extra pod beyond the desired count exists at any moment. The result is a cautious, zero-downtime rollout: all existing pods keep serving traffic while one new pod at a time is added and verified.

Q7: Switch a Deployment's strategy to Recreate and describe a concrete scenario where this is required instead of RollingUpdate.

```
strategy:   type: Recreate
```

With Recreate, ALL existing pods are terminated before any new pods are created. This causes a brief period of downtime.

Concrete scenario: a database schema migration that is not backward-compatible. If old and new application versions run simultaneously (as they would during a RollingUpdate), the new app might write data in a format the old app cannot read, causing data corruption or errors. With Recreate you guarantee only one version runs at a time.

Q8: Add CPU/memory requests and limits to a Deployment's container and verify they appear in the running pod spec.

```
resources:   requests:   cpu: "100m"   memory: "128Mi"   limits:
cpu: "250m"   memory: "256Mi"
```

Verify:

```
kubectl get pod <pod-name> -o jsonpath='{.spec.containers[0].resources}'
```

Requests inform the scheduler how much capacity is needed to place the pod. Limits enforce a hard ceiling; if the container exceeds the memory limit it is OOMKilled.

Q9: Set revisionHistoryLimit: 3 and explain how it affects your ability to roll back and how many old ReplicaSets are kept.

```
spec:  revisionHistoryLimit: 3
```

Kubernetes keeps the last 3 old (scaled-down) ReplicaSets in addition to the current one. You can roll back to any of those 3 revisions with `kubectl rollout undo --to-revision=<N>`. Revisions older than the limit are garbage-collected. The default is 10. Setting it lower saves etcd memory at the cost of losing older rollback targets.

Q10: Set a Deployment's image to a non-existent tag, observe the rollout get stuck, and explain why the old pods keep serving traffic.

```
kubectl set image deployment/web nginx=nginx:nonexistent
```

Watch the stuck rollout:

```
kubectl rollout status deployment/web # blocks indefinitely kubectl  
get pods # new pod shows ImagePullBackOff
```

The old pods keep serving traffic because of the `maxUnavailable: 0` (or default) constraint: Kubernetes will not remove an old pod until a replacement is Ready. Since the new pod never becomes Ready (it cannot pull the image), the old pods are never terminated. This is the self-healing behavior of rolling updates.

Q11: Use a label selector to list only the pods belonging to one Deployment, and explain the Deployment → ReplicaSet → Pod selector relationship.

```
kubectl get pods -l app=web
```

Selector relationship:

- The Deployment has `spec.selector.matchLabels` (e.g. `app: web`). It manages ReplicaSets whose labels match.
- Each ReplicaSet has the same selector and owns pods whose labels match.
- When a Deployment creates a ReplicaSet, it adds a `pod-template-hash` label to distinguish it from ReplicaSets of other revisions.

The chain is: Deployment selects ReplicaSet(s) → ReplicaSet selects Pods. Labels are the glue; misconfigured labels break ownership.

Q12: Expose a Deployment with `kubectl expose`, then explain what object was created and how its selector was derived.

```
kubectl expose deployment web --port=80 --target-port=80 --  
type=ClusterIP
```

A Service object named 'web' is created. Its selector is automatically copied from the Deployment's `spec.selector.matchLabels`. This means the Service routes traffic to all pods that the Deployment manages. You can inspect the result with:

```
kubectl get service web -o yaml
```

Q13: Add a sidecar (second) container to a Deployment's pod template and explain how the two containers share the pod network and volumes.

```
spec:
  template:
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
      - name: log-shipper
        image: busybox
        command: ["sh", "-c", "tail -f /var/log/nginx/access.log"]
      volumeMounts:
      - name: logs
        mountPath: /var/log/nginx
    volumes:
    - name: logs
      emptyDir: {}
```

All containers in a pod share the same network namespace: they communicate via localhost and share the same IP address and ports. They also share any volumes mounted in the pod spec. In the example above, both containers access the same log directory through the emptyDir volume.

Q14: Write a Deployment manifest for httpd:2.4 with 2 replicas, a named container port, and a nodeSelector; explain why it stays Pending if no node matches.

```
apiVersion: apps/v1 kind: Deployment metadata: name: httpd-app spec:
replicas: 2 selector: matchLabels: app: httpd-app
template: metadata: labels: app: httpd-app spec:
nodeSelector: disktype: ssd containers:
- name: httpd image: httpd:2.4 ports:
- name: http containerPort: 80
```

If no node in the cluster has the label disktype=ssd, the scheduler cannot place the pods and they remain in Pending state. `kubectl describe pod` will show a 'no nodes available to schedule pods' event under Tolerations/NodeSelector.

Q15: Create a bare Pod (no controller) running busybox that sleeps, then explain what happens when you delete it versus a Deployment-managed pod.

```
kubectl run bare-pod --image=busybox --restart=Never -- sleep 3600
```

When you delete a bare Pod (`kubectl delete pod bare-pod`), it is gone permanently. There is no controller watching it, so it is not recreated.

When you delete a Deployment-managed pod, the ReplicaSet controller detects that the actual count dropped below the desired count and immediately creates a new pod to replace it. This is the self-healing property of controllers.

Q16: Write a StatefulSet for redis:7 with 3 replicas plus a headless Service, and show the stable pod names and per-pod DNS records.

```
apiVersion: v1 kind: Service metadata: name: redis-headless spec:
clusterIP: None selector: app: redis ports:
- port: 6379 ---
apiVersion: apps/v1 kind: StatefulSet metadata: name: redis spec:
serviceName: redis-headless replicas: 3 selector: matchLabels:
app: redis template: metadata: labels: app: redis
spec: containers:
- name: redis image: redis:7
ports:
- containerPort: 6379
```

Stable pod names: redis-0, redis-1, redis-2.

Per-pod DNS records: redis-0.redis-headless.<namespace>.svc.cluster.local, and so on. The headless Service (clusterIP: None) causes the DNS to return individual pod IPs instead of a single virtual IP.

Q17: Create a DaemonSet running a busybox agent and explain why exactly one pod runs per node.

```
apiVersion: apps/v1 kind: DaemonSet metadata: name: agent spec:
selector: matchLabels: app: agent template: metadata:
labels: app: agent spec: containers: - name:
agent image: busybox command: ["sh", "-c", "while
true; do sleep 3600; done"]
```

A DaemonSet's controller ensures that exactly one pod runs on every node (that matches any optional nodeSelector or tolerations). When a new node joins the cluster, the DaemonSet controller automatically schedules a pod on it. When a node is removed, the pod is garbage-collected. Typical uses include log collectors, monitoring agents, and network plugins.

Q18: Create a Job using busybox/perl that computes something once and completes; show COMPLETIONS and read the result from logs.

```
apiVersion: batch/v1 kind: Job metadata: name: compute-pi spec:
template: spec: containers: - name: pi
image: perl command: ["perl", "-Mbignum=bpi", "-wle", "print
bpi(100)"] restartPolicy: Never
```

Check completion:

```
kubectl get job compute-pi # COMPLETIONS shows 1/1
```

Read the output:

```
kubectl logs job/compute-pi
```

Q19: Create a CronJob that prints the date every minute; show how to suspend it and how to list the Jobs it spawns.

```
apiVersion: batch/v1 kind: CronJob metadata: name: print-date spec:
schedule: "* * * * *" jobTemplate: spec: template:
spec: containers: - name: date
image: busybox command: ["date"] restartPolicy:
Never
```

Suspend the CronJob (no new Jobs are created):

```
kubectl patch cronjob print-date -p '{"spec":{"suspend":true}}'
```

List the Jobs it spawned:

```
kubectl get jobs -l job-name
```

Q20: Manually run a Job from an existing CronJob to test it on demand.

```
kubectl create job test-run --from=cronjob/print-date
```

This creates a one-off Job using the same pod template as the CronJob. Useful for testing without waiting for the next scheduled execution.

Q21: Show that a StatefulSet creates and terminates pods in order, and contrast this with a Deployment.

Scale a StatefulSet up and watch pod creation:

```
kubectl scale statefulset redis --replicas=3 kubectl get pods -w
```

Pods are created in order: redis-0 first (must be Running/Ready), then redis-1, then redis-2. Termination is in reverse order: redis-2 first.

A Deployment creates and deletes pods in parallel (no ordering guarantee). StatefulSet ordering is required for clustered stateful applications like databases where each node may need to know the previous node is healthy before joining the cluster.

Q22: Create a multi-container Pod where two containers share an emptyDir — one writes a file, the other reads it. Prove it's shared.

```
apiVersion: v1 kind: Pod metadata:  name: shared-vol spec:
containers:    - name: writer      image: busybox      command:
["sh", "-c", "echo hello > /data/msg; sleep 3600"]      volumeMounts:
- name: shared      mountPath: /data      - name: reader
image: busybox      command: ["sh", "-c", "sleep 5; cat /data/msg; sleep
3600"]      volumeMounts:      - name: shared      mountPath:
/data      volumes:      - name: shared      emptyDir: {}
```

Prove it is shared:

```
kubectl logs shared-vol -c reader # outputs: hello
```

Q23: List the StorageClasses, PVs and PVCs.

```
kubectl get storageclass kubectl get pv kubectl get pvc --all-namespaces
```

StorageClasses define how volumes are dynamically provisioned (e.g. which provisioner to use, IOPS, reclaim policy). PersistentVolumes (PVs) are cluster-level storage resources. PersistentVolumeClaims (PVCs) are namespace-scoped requests for storage that bind to a suitable PV.

Q24: Mount a ConfigMap as a volume so each key becomes a file, and verify the contents inside the pod.

```
kubectl create configmap app-config --from-literal=db_host=postgres --
from-literal=db_port=5432
# In pod spec: volumes:      - name: config-vol      configMap:      name:
app-config containers:      - name: app      volumeMounts:      - name:
config-vol      mountPath: /etc/config
```

Verify inside the pod:

```
kubectl exec <pod> -- ls /etc/config # shows: db_host db_port
kubectl exec <pod> -- cat /etc/config/db_host # shows: postgres
```

Q25: Mount a single ConfigMap key to a specific path using subPath; explain when it's needed and its update caveat.

```
volumeMounts:      - name: config-vol      mountPath: /etc/nginx/nginx.conf
subPath: nginx.conf
```

subPath is needed when you want to mount a single key into an existing directory without overwriting the entire directory contents. For example, mounting only the nginx.conf key into /etc/nginx/ alongside other existing files.

Caveat: when a ConfigMap is updated, volumes mounted with subPath are NOT automatically updated in running pods. You must restart the pod to pick up the new value. Volumes mounted without subPath update automatically (with a short delay).

Q26: Mount a Secret as a volume and verify the files contain decoded values with restrictive permissions.

```
volumes:      - name: secret-vol      secret:      secretName: my-secret
defaultMode: 0400 containers:      - volumeMounts:      - name: secret-vol
mountPath: /etc/secret
```

Verify:

```
kubectl exec <pod> -- ls -la /etc/secret # permissions: -r-----
kubectl exec <pod> -- cat /etc/secret/username # decoded plain text
```

Kubernetes stores Secret values as base64 in etcd. When mounted as a volume, it automatically decodes them to plain text. The restrictive permissions (0400) prevent other processes or users from reading the files.

Q27: Show that scaling a StatefulSet creates one PVC per replica, then explain what happens to those PVCs when the StatefulSet is deleted.

```
kubectl get pvc -l app=redis # Shows: data-redis-0, data-redis-1, data-redis-2
```

When a StatefulSet is deleted, the PVCs it created are NOT automatically deleted. This is intentional — the data is preserved in case you want to recreate the StatefulSet and reattach the volumes. You must delete the PVCs manually if you want to remove the data:

```
kubectl delete pvc -l app=redis
```

Q28: Use an initContainer to pre-populate data into a shared volume before the main container reads it.

```
initContainers:  - name: init-data      image: busybox      command:
["sh", "-c", "echo 'initialized' > /data/ready"]      volumeMounts:      -
name: shared      mountPath: /data containers:  - name: main
image: busybox      command: ["sh", "-c", "cat /data/ready; sleep 3600"]
volumeMounts:      - name: shared      mountPath: /data volumes:  -
name: shared      emptyDir: {}
```

Init containers always complete before any regular containers start. This guarantees the main container finds the data ready.

Q29: Set readOnly: true on a volume mount and prove writes are rejected.

```
volumeMounts:  - name: data      mountPath: /mnt/data      readOnly: true
```

Prove writes are rejected:

```
kubectl exec <pod> -- touch /mnt/data/test # Output: touch:
/mnt/data/test: Read-only file system
```

Q30: Set a sizeLimit on an emptyDir and explain what happens if the container exceeds it.

```
volumes:  - name: tmp      emptyDir:      sizeLimit: 100Mi
```

If the container writes more than 100Mi to the emptyDir, the kubelet evicts the pod. The eviction is triggered by the container disk pressure monitoring. This prevents a single pod from consuming all disk space on a node.

Q31: Create a generic Secret from literals for a DB username/password, then show the stored values are base64-encoded, not encrypted.

```
kubectl create secret generic db-creds \      --from-literal=username=admin
\      --from-literal=password=s3cr3t
```

Inspect the raw stored value:

```
kubectl get secret db-creds -o jsonpath='{.data.username}' | base64 -d #
Output: admin
```

The value is only base64-encoded, which is an encoding scheme, not encryption. Anyone with RBAC permission to read Secrets can decode the values immediately. For true encryption, you need Encryption at Rest configured in the API server, or an external secrets manager such as HashiCorp Vault.

Q32: Create a Secret from files (--from-file) holding a certificate and key, and identify the resulting keys.

```
kubectl create secret generic tls-files \    --from-  
file=tls.crt=/path/to/cert.pem \    --from-file=tls.key=/path/to/key.pem
```

The resulting keys in the Secret are `tls.crt` and `tls.key` (the filenames you specified). When mounted as a volume, these become the file names inside the mount directory.

Q33: Create a `kubernetes.io/tls` typed Secret from a cert/key pair and explain where it's consumed.

```
kubectl create secret tls my-tls \    --cert=cert.pem \    --key=key.pem
```

This creates a Secret of type `kubernetes.io/tls` with keys `tls.crt` and `tls.key`. It is consumed by Ingress controllers and other components that need to terminate TLS connections. The Ingress spec references it in the `tls.secretName` field.

Q34: Mount a Secret as a volume and explain the security trade-offs versus env vars.

Volume-mounted Secret advantages:

- Values are stored in a tmpfs (in-memory filesystem), not written to disk.
- Secrets can be automatically updated when the ConfigMap or Secret changes (without subPath).
- The file permissions can be restricted (e.g. 0400).

Environment variable disadvantages:

- Env vars are visible in `kubectl describe pod` output.
- Child processes inherit env vars, increasing the attack surface.
- Env vars do not update automatically; the pod must be restarted.

Volume mounts are generally considered more secure for sensitive values.

Q35: Use `envFrom` with a `secretRef` to load every key of a Secret as env vars at once.

```
envFrom:    - secretRef:        name: db-creds
```

Every key in the Secret becomes an environment variable with the same name and decoded value. This is convenient when a Secret has many keys, but be aware of the security trade-offs mentioned above (all keys are exposed as env vars).

Q36: Create a `docker-registry` (image-pull) Secret and reference it via `imagePullSecrets`; explain when it's required.

```
kubectl create secret docker-registry my-pull-secret \    --docker-  
server=quay.io \    --docker-username=<USER> \    --docker-  
password=<TOKEN>  
# In pod spec: imagePullSecrets:    - name: my-pull-secret
```

This is required when pulling images from a private registry that requires authentication. Without it, the kubelet cannot authenticate and the pod enters `ImagePullBackOff` state.

Q37: Create a Secret with several keys and selectively mount only one of them into a pod.

```
volumes: - name: secret-vol      secret:      secretName: db-creds
items:   - key: password         path: db-password
```

Only the 'password' key is mounted, available at /etc/secret/db-password. The 'username' key is not exposed in the filesystem. The items array lets you select specific keys and rename them when mounted.

Q38: Create a ClusterIP Service for a Deployment and resolve it by DNS from another pod.

```
kubectl expose deployment web --port=80 --type=ClusterIP
```

From another pod, resolve the Service by DNS:

```
kubectl run debug --rm -it --image=busybox -- nslookup
web.default.svc.cluster.local
```

The FQDN format is <service-name>.<namespace>.svc.cluster.local. CoreDNS resolves this to the ClusterIP of the Service, which then load-balances to the backing pods.

Q39: Create a NodePort Service and reach it via minikube service and minikube ip.

```
kubectl expose deployment web --port=80 --type=NodePort
```

Access via minikube service:

```
minikube service web --url
```

Access via node IP and port:

```
curl $(minikube ip):<nodePort>
```

NodePort allocates a port in the 30000-32767 range on every cluster node. Traffic arriving on that port is forwarded to the Service, which then routes to a pod.

Q40: Create a LoadBalancer Service, explain why EXTERNAL-IP is <pending> on minikube, and obtain access with minikube tunnel.

```
kubectl expose deployment web --port=80 --type=LoadBalancer
```

On a cloud provider, the cloud controller manager provisions an external load balancer and populates EXTERNAL-IP. Minikube runs locally and has no cloud controller, so EXTERNAL-IP stays <pending>.

Fix with minikube tunnel (run in a separate terminal):

```
minikube tunnel
```

This creates a network route and assigns a local IP to the LoadBalancer Service so you can reach it from your host.

Q41: Create a headless Service (clusterIP: None) for a StatefulSet and show it returns per-pod A records instead of a single VIP.

```
apiVersion: v1 kind: Service metadata:  name: redis-headless spec:
clusterIP: None selector:      app: redis  ports:      - port: 6379
```

Verify DNS returns individual pod IPs:

```
kubectl run debug --rm -it --image=busybox -- nslookup redis-headless #
Returns multiple A records, one per pod
```

A headless Service bypasses kube-proxy and the virtual IP layer. Clients receive the IP addresses of all matching pods directly, which is required for stateful applications that need to address specific replicas.

Q42: Inspect a Service's Endpoints/EndpointSlice and explain how they're populated from the pod selector.

```
kubectl get endpoints web kubectl get endpointslices -l
kubernetes.io/service-name=web
```

The Endpoints/EndpointSlice object is automatically managed by the endpoints controller. It watches for pods that match the Service selector AND are in a Ready state. Each Ready pod's IP and the target port are added as an endpoint. If a pod is not Ready (failing readiness probe), its IP is removed, stopping traffic from being sent to it.

Q43: Demonstrate cross-namespace access using the FQDN, and show the short name fails from another namespace.

From a pod in namespace 'staging', access a Service in namespace 'production':

```
# FQDN works: nslookup web.production.svc.cluster.local # Short name
fails (resolves in current namespace 'staging'): nslookup web # Returns:
server can't find web
```

The short name only resolves within the same namespace because the DNS search path only appends the pod's own namespace suffix.

Q44: Compare kubectl port-forward to a Service versus to a Pod.

Port-forward to a Service:

```
kubectl port-forward service/web 8080:80
```

Port-forward to a Pod:

```
kubectl port-forward pod/web-abc123 8080:80
```

To a Service: kubectl randomly selects one of the Service's endpoints on every connection. The Service selector and load-balancing apply. Use this for general Service debugging.

To a Pod: all connections go to that specific pod, bypassing the Service entirely. Use this for debugging a single pod in isolation.

Neither is suitable for production; both tunnel through the Kubernetes API server.

Q45: Explain the difference between a Service's port, targetPort, and nodePort.

- port: the port exposed by the Service (ClusterIP). Other pods connect on this port.
- targetPort: the port the container actually listens on. Traffic is forwarded from port to targetPort.
- nodePort: the port opened on every node's external interface (only for NodePort/LoadBalancer Services). External traffic enters here and is forwarded to port.

Example: port 80 → targetPort 8080 → nodePort 30080. External clients use port 30080; internal clients use port 80; the container listens on 8080.

Q46: Verify connectivity to a ClusterIP Service from a throwaway debug pod with wget/nc.

```
kubectl run tmp --rm -it --image=busybox -- sh # Inside the shell: wget
-qO- http://web.default.svc.cluster.local nc -zv web 80
```

This is the standard technique for verifying Service reachability from inside the cluster without modifying existing workloads.

Q47: Create two Deployments and one Service that load-balances across both by sharing a common label; prove requests hit both.

```
# Deployment A
kubectl create deployment app-a --image=nginx:1.25
kubectl label deployment app-a tier=frontend
kubectl label pods -l app=app-a tier=frontend
# Deployment B
kubectl create deployment app-b --image=nginx:1.27
kubectl label pods -l app=app-b tier=frontend
# Service selecting on the shared label
kubectl create service clusterip frontend --tcp=80:80
kubectl patch service frontend -p '{"spec":{"selector":{"tier":"frontend"}}}'
```

Prove requests hit both pods by checking logs on each pod while sending multiple requests.

Q48: Systematically diagnose why a pod can't reach a Service: DNS → endpoints → selector labels → readiness → port.

- DNS: `kubectl exec <pod> -- nslookup <svc>.<ns>.svc.cluster.local` — confirms the Service exists and CoreDNS is working.
- Endpoints: `kubectl get endpoints <svc>` — if empty, no pods match the selector or none are Ready.
- Selector labels: `kubectl get pods --show-labels` — compare with Service spec.selector.
- Readiness: `kubectl describe pod <pod>` — look for failing readiness probes in Events.
- Port: confirm container port matches Service targetPort with `kubectl get service <svc> -o yaml`.

Q49: Expose a service with a NodePort and verify it works.

```
kubectl expose deployment web --type=NodePort --port=80
NODE_PORT=$(kubectl get svc web -o jsonpath='{.spec.ports[0].nodePort}')
curl $(minikube ip):$NODE_PORT
```

A successful HTTP response confirms the NodePort Service is routing traffic correctly to the pods.

Q50: Add an HTTP livenessProbe hitting / on port 80 to an nginx Deployment and verify via kubectl describe pod.

```
livenessProbe:  httpGet:      path: /      port: 80
initialDelaySeconds: 10  periodSeconds: 5
```

Verify:

```
kubectl describe pod <pod-name> # Look for: Liveness: http-get
http://:80/ delay=10s timeout=1s period=5s
```

If the probe fails three consecutive times (failureThreshold default), the kubelet restarts the container.

Q51: Add a tcpSocket readiness probe to a redis container on port 6379 and verify it.

```
readinessProbe:  tcpSocket:      port: 6379  initialDelaySeconds: 5
periodSeconds: 10
```

Verify:

```
kubectl describe pod <pod-name> # Look for: Readiness: tcp-socket :6379
delay=5s timeout=1s period=10s
```

A pod that fails its readiness probe is removed from the Service endpoints and receives no traffic, but is not restarted.

Q52: Configure a startup probe with a `failureThreshold * periodSeconds` budget for a ~2-minute boot and justify the numbers.

```
startupProbe:  httpGet:      path: /health      port: 8080
failureThreshold: 24   periodSeconds: 5
```

Justification: $24 \text{ failures} \times 5 \text{ seconds} = 120 \text{ seconds (2 minutes)}$. During this window, the liveness and readiness probes are disabled. If the app has not become healthy within 2 minutes, the container is killed. This prevents the liveness probe from killing a slow-starting container prematurely.

Q53: Explain the default behavior when no probes are defined at all.

When no probes are defined:

- Liveness: Kubernetes assumes the container is always alive as long as the main process (PID 1) is running. If the process exits with a non-zero code, the container is restarted.
- Readiness: Kubernetes assumes the container is Ready as soon as it starts, and immediately adds it to the Service endpoints.
- Startup: Not applicable; liveness and readiness apply immediately.

This means a slow-starting app might receive traffic before it is actually ready to serve it, potentially causing errors.

LO5 — Solve problems with application shipping by using containers

Q54: `podman run -d --name web -p 80:8080 docker.io/library/nginx` — what's reversed?

Mistake: the port mapping is reversed. The format is `-p <host_port>:<container_port>`. nginx listens on port 80 inside the container, so the correct mapping is `-p 8080:80` (or `-p 80:80` if you want the same port on the host). As written, the host port 80 maps to container port 8080, but nothing is listening on 8080 in the nginx container, so requests to localhost:80 get no response.

Fix:

```
podman run -d --name web -p 8080:80 docker.io/library/nginx
```

Q55: `podman run -d --name db -e MYSQL_ROOT_PASSWORD docker.io/library/mysql` — the container exits during init.

Mistake: `MYSQL_ROOT_PASSWORD` is specified as a flag name without a value. The environment variable is set to an empty string. MySQL requires a non-empty root password (or `MYSQL_ALLOW_EMPTY_PASSWORD=yes` or `MYSQL_RANDOM_ROOT_PASSWORD=yes`). Without a valid password, the entrypoint exits with an error.

Fix:

```
podman run -d --name db -e MYSQL_ROOT_PASSWORD=my_secure_password
docker.io/library/mysql
```

Q56: podman run -d --name app --network host -p 8080:80 docker.io/library/nginx — why is -p effectively ignored?

Mistake: `--network host` makes the container share the host's network namespace directly. There is no network isolation and no port translation layer. The `-p` flag is only meaningful for bridge networks. With host networking, nginx's port 80 is already bound directly on the host as port 80, not 8080. The `-p` flag is silently ignored.

Fix: either remove `--network host` and use bridge networking with `-p`, or access nginx on port 80 directly (and remove `-p`):

```
podman run -d --name app -p 8080:80 docker.io/library/nginx
```

Q57: podman run -d --name c1 docker.io/library/busybox — goes straight to Exited (0).

Mistake: the busybox image's default command is `sh`. When run without a terminal attached (`-d`, detached), `sh` has no input and exits immediately with code 0.

Fix — provide a long-running command:

```
podman run -d --name c1 docker.io/library/busybox sleep infinity
```

Q58: podman run --rm -d --name job docker.io/library/alpine echo hello — then podman logs job fails.

Mistake: `--rm` removes the container as soon as it exits. Since 'echo hello' completes almost instantly, the container is deleted before you can run `podman logs`. The combination of `--rm` and `-d` is valid but means the container disappears immediately after the command completes.

Fix — remove `--rm` if you want to inspect logs:

```
podman run -d --name job docker.io/library/alpine echo hello podman logs
job
```

Q59: podman run -d -p 8080:80 --memory 8m docker.io/library/mysql — the database never becomes healthy.

Mistake: 8 MiB is far too little memory for MySQL. MySQL requires at least 300–400 MiB at startup just for its InnoDB buffer pool and internal data structures. With only 8m, the process is OOM-killed by the kernel almost immediately.

Fix — provide a realistic memory limit:

```
podman run -d -p 3306:3306 --memory 512m -e MYSQL_ROOT_PASSWORD=pass
docker.io/library/mysql
```

Q60: podman exec a ping b — name resolution fails on the default network.

Mistake: on Podman's default bridge network, containers cannot resolve each other by name. DNS-based name resolution only works on user-defined networks.

Fix — create a user-defined network and attach both containers to it:

```
podman network create mynet podman run -d --name a --network mynet
docker.io/library/alpine sleep 1d podman run -d --name b --network mynet
docker.io/library/alpine sleep 1d podman exec a ping b
```

Q61: podman run -d --name web -v ./html:/usr/share/nginx/html docker.io/library/nginx — files not visible or SELinux denies access.

Mistake: on SELinux-enabled systems (RHEL, Fedora, CentOS), the host directory needs an SELinux context that allows container access. The :z (shared) or :Z (private, unshared) relabeling flag is missing.

Fix:

```
podman run -d --name web -v ./html:/usr/share/nginx/html:Z
docker.io/library/nginx
```

:Z applies a private SELinux label to the volume, giving only this container access. :z applies a shared label, usable by multiple containers.

Q62: FROM debian:12 / RUN apt-get install -y nginx — build fails to find the package.

Mistake: apt-get install requires the package cache to be populated first. A fresh Debian image has an empty cache. Without apt-get update, apt cannot find any packages.

Fix:

```
FROM debian:12 RUN apt-get update && apt-get install -y nginx
```

Q63: CMD python app.py (shell form) — app starts but doesn't handle signals.

Mistake: shell form CMD python app.py runs as /bin/sh -c 'python app.py'. The shell process becomes PID 1 and does not forward signals (SIGTERM) to the Python process. This means docker stop or Kubernetes graceful shutdown does not reach the application.

Fix — use exec form so Python is PID 1 directly:

```
CMD ["python", "app.py"]
```

Q64: COPY ./app / WORKDIR /app / RUN npm install — every source change triggers a full npm install.

Mistake: COPY ./app copies all source files before npm install. Docker's layer cache is invalidated whenever any source file changes, forcing npm install to re-run every time.

Fix — copy only package.json first, install dependencies, then copy the rest:

```
FROM node:20 WORKDIR /app COPY package.json package-lock.json ./ RUN npm
install COPY . .
```

Q65: FROM alpine:3.20 / ENV PATH=/app/bin / RUN apk add --no-cache curl — curl/sh not found.

Mistake: ENV PATH=/app/bin replaces the entire PATH variable instead of prepending to it. The standard system paths (/usr/local/bin, /usr/bin, /bin) are lost, so the shell cannot find curl or any other standard tool.

Fix — prepend to the existing PATH:

```
ENV PATH=/app/bin:$PATH
```

Q66: EXPOSE 8080 / CMD ["python3", "-m", "http.server", "3000"] — -p 8080:8080 but nothing answers.

Mistake: the server starts on port 3000 but EXPOSE declares port 8080. These must match. EXPOSE is documentation only — it does not bind any port. The -p flag maps host port 8080 to container port 8080, but nothing is listening on 8080 inside the container.

Fix — make port consistent:

```
EXPOSE 3000 CMD ["python3", "-m", "http.server", "3000"]
```

Q67: RUN apt-get update && apt-get install -y build-essential — image is huge. How to cut size in the same layer?

Fix — clean up the apt cache in the same RUN layer:

```
RUN apt-get update && \ apt-get install -y build-essential && \
rm -rf /var/lib/apt/lists/*
```

Cleaning in the same RUN layer is critical. If you clean in a separate RUN, the cache files still exist in the earlier layer and Docker includes them in the image. Combining with && ensures the cleanup happens before Docker commits the layer.

Q68: FROM python:3.11 / USER appuser / COPY ... / RUN pip install — permission denied errors.

Mistake: USER appuser is set before COPY, so the copied files are owned by root (COPY always runs as root, ignoring the current USER). Then pip install, running as appuser, cannot read the files.

Fix — copy and install as root, then switch user:

```
FROM python:3.11 COPY requirements.txt /app/requirements.txt RUN pip
install -r /app/requirements.txt USER appuser
```

Q69: FROM golang:1.22 COPY . /src / RUN go build / CMD ["/src/app"] — final image is ~1 GB. How does a multi-stage build fix it?

The golang base image includes the full Go toolchain and is ~800 MB. The compiled binary is typically a few MB.

Fix with a multi-stage build:

```
FROM golang:1.22 AS builder COPY . /src WORKDIR /src RUN go build -o app
. FROM scratch COPY --from=builder /src/app /app CMD ["/app"]
```

The final image contains only the binary, reducing the size from ~1 GB to a few MB. The build toolchain never ships to production.

Q70: A pod is stuck Pending. Walk through the diagnosis.

```
kubectl describe pod <pod-name>
```

Look at the Events section at the bottom. Common reasons and their messages:

- Insufficient cpu/memory: no node has enough capacity. Fix: reduce resource requests or add nodes.
- 0/1 nodes are available: nodeSelector or affinity rules do not match any node. Fix: add the required label to a node.
- persistentvolumeclaim not found or Pending: PVC is unbound. Fix: check StorageClass and PV availability.
- no nodes available to schedule pods: taints on all nodes and no tolerations. Fix: add tolerations to the pod spec.

Q71: Find a deployment from older tasks, remove spaces, try to deploy it. Identify the cause.

Removing spaces in a YAML file usually breaks indentation, which is syntactically significant in YAML. `kubectl apply` will return an error such as:

```
error: error parsing manifest.yaml: error converting YAML to JSON: yaml:
line N: ...
```

Use a YAML linter or `--dry-run=server` to find the problem before applying:

```
kubectl apply -f manifest.yaml --dry-run=server
```

Fix by restoring proper indentation with 2-space increments.

Q72: A pod is in ImagePullBackOff. Diagnose and fix.

```
kubectl describe pod <pod-name> # Events will show: Failed to pull image
... 404 Not Found, etc.
```

Common causes and fixes:

- Image typo / wrong tag: check the exact image name and tag in Docker Hub or the registry. Fix the image field in the manifest.
- Private registry without pull secret: create a docker-registry Secret and add it to imagePullSecrets.
- Incorrect registry URL: verify the full registry path (e.g. quay.io/myorg/myimage:tag).

Q73: A pod is in CrashLoopBackOff. Use `kubectl logs --previous` to read the last crash and fix.

```
kubectl logs <pod-name> --previous
```

The `--previous` flag retrieves the logs from the terminated container instance (the one that crashed). Common reasons:

- Application error at startup: fix the code or configuration.
- Missing environment variable: add the required env var.
- Wrong command/args: correct the CMD/ENTRYPOINT.

After fixing, redeploy the Deployment or delete and recreate the pod.

Q74: A pod's last state shows OOMKilled. Confirm it and fix.

```
kubectl get pod <pod> -o yaml | grep -A5 lastState #
lastState.terminated.reason: OOMKilled kubectl describe pod <pod> #
Shows: OOMKilled in the Last State section
```

Fix — raise the memory limit:

```
resources:    limits:    memory: "512Mi"
```

Alternatively, profile the application and fix the memory leak if one exists.

Q75: A Deployment's pods never become Ready. Trace it to a failing readiness probe.

```
kubectl describe pod <pod-name> # Events: Readiness probe failed: ...
```

Common readiness probe failures:

- Wrong port: app listens on 8080, probe checks port 80.
- Wrong path: probe checks /health, app exposes /healthz.
- App not ready within initialDelaySeconds: increase the delay.

Fix the probe or fix the application to match the probe.

Q76: A Service returns nothing. Diagnose a Service/pod label mismatch.

```
kubectl get endpoints <service-name>
```

If the Endpoints object shows <none>, the Service selector does not match any running pod. Compare:

```
kubectl get service <svc> -o jsonpath='{.spec.selector}' kubectl get pods --show-labels
```

Fix the label on the pod or the selector on the Service so they match.

Q77: `spec.selector.matchLabels` doesn't match `spec.template.metadata.labels` — explain the error and fix.

kubectl apply returns an error like:

```
The Deployment 'web' is invalid: spec.template.metadata.labels: Invalid value: ... selector does not match template labels
```

A Deployment's selector must be a subset of the pod template's labels; otherwise the Deployment can never own its own pods. Fix by ensuring they match:

```
selector:      matchLabels:      app: web template:      metadata:      labels:
app: web      # must contain all selector keys
```

Q78: `resources.requests` exceed any node's capacity — explain Pending and fix.

If a pod requests more CPU or memory than any single node can provide, the scheduler cannot place it anywhere. kubectl describe pod shows:

```
Events: 0/1 nodes are available: Insufficient cpu.
```

Fix — reduce requests to a value that fits on available nodes:

```
resources:      requests:      cpu: "100m"      memory: "128Mi"
```

Q79: Given a pod mounting a PVC that doesn't exist, diagnose the FailedMount/Pending and create the PVC.

```
kubectl describe pod <pod> # Events: persistentvolumeclaim not bound
kubectl get pvc <pvc-name> # NotFound
```

Create the missing PVC:

```
apiVersion: v1 kind: PersistentVolumeClaim metadata:      name: my-pvc
spec:      accessModes: [ReadWriteOnce]      resources:      requests:
storage: 1Gi
```

Q80: A container based on ubuntu with no long-running command shows Completed/CrashLoopBackOff. Explain and fix.

ubuntu's default CMD is /bin/bash. When run without an interactive terminal (in a pod), bash has no input and exits immediately with code 0. The pod controller sees the exit and restarts it, creating a loop.

Fix — add a long-running command:

```
command: ["sleep", "infinity"]
```

Q81: Use kubectl get events --sort-by to triage a pod.

```
kubectl get events --sort-by=.lastTimestamp -n <namespace> # or with
newer kubectl: kubectl events --for pod/<pod-name>
```

Events show the chronological history of what happened to a resource: scheduling decisions, image pulls, probe failures, container starts and restarts. Sorting by lastTimestamp shows the most recent activity at the bottom.

Q82: Use `kubectl exec -it` to debug DNS with `nslookup` and `cat /etc/resolv.conf`.

```
kubectl exec -it <pod> -- sh nslookup
kubernetes.default.svc.cluster.local cat /etc/resolv.conf
```

`/etc/resolv.conf` shows the nameserver (CoreDNS ClusterIP), the search domains, and `ndots` settings. If `nslookup` fails, CoreDNS may be down or the pod's DNS configuration is incorrect.

Q83: Use an ephemeral debug container to troubleshoot a no-shell/distroless container.

```
kubectl debug -it <pod> --image=busybox --target=<container-name>
```

Distroless or scratch-based images have no shell or debugging tools. An ephemeral container is injected into the running pod, sharing the same network namespace and (optionally) the same process namespace as the target container. This lets you run debugging commands without modifying the original image.

Q84: Spin up a throwaway pod to test connectivity to a Service from inside the cluster.

```
kubectl run tmp --rm -it --image=busybox -- wget -qO-
http://web.default.svc.cluster.local
```

`--rm` deletes the pod after the session ends, keeping the cluster clean. This is the standard way to test Service connectivity without deploying a permanent debug pod.

Q85: A node shows `NotReady`. What would you check?

- Run `kubectl describe node <node-name>` — look for conditions: `DiskPressure`, `MemoryPressure`, `PIDPressure`, `NetworkUnavailable`.
- Check kubelet status on the node: `sudo systemctl status kubelet`.
- Check kubelet logs: `sudo journalctl -u kubelet -n 100`.
- Check CNI plugin: if the network plugin (e.g. Calico, Flannel) is not running, pods cannot communicate.
- On minikube: minikube logs and `kubectl describe node minikube`.

Q86: A pod is stuck `Terminating`. Explain finalizers and `force-delete`.

Finalizers are markers in an object's metadata that prevent deletion until certain cleanup operations complete (e.g. releasing a cloud load balancer, deleting associated volumes). If a finalizer's controller is broken, the pod may be stuck in `Terminating` indefinitely.

Force-delete (use with caution):

```
kubectl delete pod <pod> --grace-period=0 --force
```

Risk: the pod process may still be running on the node. The kubelet will eventually clean it up, but there could be a brief window where two instances of the pod are active.

Q87: Given a manifest with wrong `apiVersion/kind` pairing (e.g. `apps/v1 + Pod`), explain the error.

`kubectl apply` returns:

```
error: resource mapping not found for name ...: no matches for kind
"Pod" in group "apps"
```

Pods belong to the core API group, not the apps group. The correct apiVersion for a Pod is v1, not apps/v1. Fix:

```
apiVersion: v1 kind: Pod
```

apps/v1 is for higher-level controllers: Deployment, ReplicaSet, StatefulSet, DaemonSet.

Q88: A pod fails with CreateContainerConfigError because a configMapKeyRef key is missing. Diagnose and fix.

```
kubectl describe pod <pod> # Events: Error: couldn't find key db_host in ConfigMap default/app-config
```

The pod spec references a key that does not exist in the named ConfigMap. Fix — either add the missing key to the ConfigMap:

```
kubectl create configmap app-config --from-literal=db_host=postgres
```

Or correct the key name in the pod spec's env section.

Q89: A pod can't mount a Secret that lives in a different namespace. Explain namespace scoping and fix.

Secrets (and ConfigMaps) are namespace-scoped resources. A pod can only reference Secrets in its own namespace. There is no cross-namespace Secret reference.

Fix — recreate the Secret in the pod's namespace:

```
kubectl get secret my-secret -n source-ns -o yaml | \ sed  
's/namespace: source-ns/namespace: target-ns/' | \ kubectl apply -f -
```

Q90: A rollout is stuck with ProgressDeadlineExceeded.

```
kubectl rollout status deployment/web # Output: Waiting for deployment  
'web' rollout to finish ... # error: progress deadline exceeded kubectl  
describe deployment web # Events: ReplicaSet never available within  
progress deadline
```

The Deployment's spec.progressDeadlineSeconds (default 600 s) elapsed before the new ReplicaSet became fully available. Common causes: ImagePullBackOff, failing readiness probes, insufficient resources.

Fix the root cause (check pod events and logs), then rollback if needed:

```
kubectl rollout undo deployment/web
```

Q91: Catch indentation/field typos before applying with --dry-run=server.

```
kubectl apply -f manifest.yaml --dry-run=server # or validate only:  
kubectl apply -f manifest.yaml --validate=true --dry-run=client
```

--dry-run=server sends the manifest to the API server for full validation (schema check, admission controllers) without persisting any changes. --dry-run=client validates locally. Both catch typos like imagePullpolicy (should be imagePullPolicy) before they cause runtime failures.

Q92: An app's logs say it can't reach its database Service. Verify in order.

- Pod running: kubectl get pods — all pods in Running state?
- Service exists: kubectl get service db — is the Service present in the correct namespace?
- Endpoints populated: kubectl get endpoints db — are pod IPs listed? If empty, selector mismatch.

- DNS resolves: `kubectl exec <app-pod> -- nslookup db.<ns>.svc.cluster.local`
- Correct port: `kubectl get service db -o yaml` — does `targetPort` match what the container listens on?

Work through each layer; the first failure reveals the broken link.

Q93: Read container `restartCount` and `lastState` with `kubectl get pod -o jsonpath`.

```
kubectl get pod <pod> -o
jsonpath='{.status.containerStatuses[0].restartCount}' kubectl get pod
<pod> -o jsonpath='{.status.containerStatuses[0].lastState}'
```

`lastState.terminated` shows reason (OOMKilled, Error), exit code, and timestamps of the most recent crash. A high `restartCount` combined with OOMKilled indicates a memory limit that is too low or a memory leak.

Q94: Compare OpenShift Route to Kubernetes Ingress.

Ingress is the standard Kubernetes API for exposing HTTP/HTTPS services. It requires an Ingress Controller (nginx, Traefik, etc.) to be installed separately. Ingress supports basic host/path routing and TLS termination.

OpenShift Route is an OpenShift-specific extension that predates Ingress. It is implemented by the built-in HAProxy router. Routes support additional features such as edge/passthrough/re-encrypt TLS termination modes and traffic splitting (A/B deployments) natively in the Route spec.

OpenShift 4.x also supports the standard Ingress API for compatibility, but Routes remain the primary mechanism and expose more OpenShift-specific capabilities.

LO6 — Evaluate the use of selected container orchestration systems

Q95: Analyze the networking model of Kubernetes versus Docker's network.

Kubernetes requires that every pod gets its own IP address and that all pods can communicate with each other without NAT. This flat network is implemented by CNI plugins (Calico, Flannel, Cilium). There is no notion of 'linking' containers; Services provide stable DNS and load balancing across pods.

Docker's default network uses a private bridge (docker0). Containers on the same host can reach each other; communication across hosts requires extra setup (overlay networks, port publishing). Docker Swarm adds a routing mesh, but it is less flexible than Kubernetes's networking model.

Q96: Evaluate Kubernetes storage abstraction (CSI, StorageClasses, dynamic provisioning) against Podman's storage plugins.

Kubernetes uses the Container Storage Interface (CSI) as a standard plugin API. StorageClasses enable dynamic provisioning: a PVC triggers automatic creation of a PV from the cloud or storage backend. This abstraction decouples workloads from the underlying storage technology and supports advanced features such as snapshots, resizing, and topology-aware provisioning.

Podman relies on simple volume plugins and host-local storage. It lacks built-in dynamic provisioning or a cluster-wide storage abstraction. For stateful workloads that need scalable, resilient storage, Kubernetes is significantly more flexible.

Q97: Evaluate the operator pattern (CRDs + controllers) for managing stateful services.

An Operator encodes operational knowledge about a specific application (e.g. Kafka, PostgreSQL) in a controller. It watches custom resources and automates tasks such as provisioning, scaling, backup, failover, and upgrades — tasks that would require manual intervention with plain manifests.

Plain Kubernetes manifests are simpler to understand but require the operator team to handle Day-2 operations manually. The Operator pattern is superior for complex stateful services where the lifecycle management logic is non-trivial, at the cost of increased initial complexity.

Q98: Assess the developer experience of plain Kubernetes versus OpenShift S2I and developer console.

Plain Kubernetes (kubectl + manifests) is flexible and widely understood, but requires developers to write and maintain YAML, understand networking primitives, and set up their own CI/CD pipelines. The learning curve is steep.

OpenShift's Source-to-Image (S2I) lets developers push source code and get a running container without writing a Dockerfile. The developer console provides a web UI for deploying apps, viewing logs, and managing builds. OpenShift optimizes for developer productivity and operational consistency in enterprise environments, at the cost of flexibility and vendor dependency.

Q99: Critically evaluate migrating a workload from OpenShift to Kubernetes.

Workloads that use only standard Kubernetes primitives (Deployments, Services, Ingress) migrate relatively easily. However, OpenShift-specific resources — Routes, BuildConfigs, ImageStreams, DeploymentConfigs, Projects, SCCs — have no direct Kubernetes equivalent and must be replaced or reworked.

Key challenges: replacing Routes with an Ingress controller, replacing S2I pipelines with Dockerfile-based builds, adjusting security contexts (OpenShift's restricted SCC prevents running as root by default), and replacing Projects with Namespaces with equivalent RBAC.

The migration effort is moderate to high, depending on how deeply OpenShift-specific APIs are used.

Q100: Evaluate running CI/CD build agents inside Kubernetes versus on dedicated VMs.

Inside Kubernetes: build agents (Jenkins agents, GitLab runners, Tekton tasks) run as pods, enabling autoscaling to zero and dynamic provisioning of per-job environments. This reduces idle infrastructure cost and improves isolation between jobs.

Challenges: noisy-neighbor effects if resource requests/limits are not set carefully; privilege escalation risk when builds need Docker-in-Docker; longer startup times for complex build pods.

Dedicated VMs offer predictable performance and simpler tooling but incur fixed costs and limited autoscaling. For most teams, Kubernetes-native CI/CD agents offer a better cost-to-performance trade-off at scale.

Q101: How does Kubernetes integrate with cloud environments and dynamic capacity provisioning?

Kubernetes integrates with cloud providers through the Cloud Controller Manager (CCM), which implements cloud-specific logic for: load balancer provisioning (LoadBalancer Services), node lifecycle management, and storage provisioning (CSI drivers).

The Cluster Autoscaler monitors unschedulable pods and scales node groups in AWS, GCP, and Azure by calling cloud provider APIs. Combined with Horizontal and Vertical Pod Autoscalers, Kubernetes can dynamically match compute capacity to application demand, which is not easily achievable on bare metal.

Q102: Critically evaluate the default security and hardening of OpenShift versus vanilla Kubernetes.

Vanilla Kubernetes ships with minimal security defaults: no pod security policies enforced by default (PSP was deprecated and removed), containers can run as root unless explicitly prevented, and the API server has broad default permissions.

OpenShift applies Security Context Constraints (SCCs) by default. The restricted SCC prevents running as root and dropping Linux capabilities, applying to all user workloads by default. OpenShift also includes built-in RBAC, audit logging, and image scanning. These defaults are why regulated industries (finance, healthcare, government) often choose OpenShift: the security posture is hardened out of the box rather than requiring post-installation configuration.

Q103: Critically evaluate the learning curve of OpenShift versus plain Kubernetes.

Plain Kubernetes has a steep learning curve due to the number of primitives (pods, deployments, services, ingress, RBAC, etc.) and the minimal default tooling. However, all skills transfer universally across cloud providers and distributions.

OpenShift adds another layer of abstraction with its own concepts (Projects, Routes, SCCs, OperatorHub). For teams new to containers, OpenShift's developer console and S2I can reduce the initial friction. However, teams must eventually understand both Kubernetes fundamentals and OpenShift extensions, making the total surface area larger. For experienced Kubernetes users, OpenShift's added abstractions can feel restrictive.

Q104: Compare the resource footprint of full Kubernetes versus k3s.

A minimal single-node Kubernetes cluster (kubeadm) requires ~1.5–2 GB RAM just for the control plane (etcd, kube-apiserver, kube-controller-manager, kube-scheduler). Full Kubernetes is designed for multi-node, highly available production clusters.

k3s is a lightweight Kubernetes distribution by Rancher. It combines all control plane components into a single binary (~70 MB), replaces etcd with SQLite by default, and requires as little as 512 MB RAM. It is ideal for edge computing, IoT, CI environments, and small on-premise deployments where full Kubernetes would be overkill.

Q105: Defend whether a small team should use Docker Compose on a single host or move straight to Kubernetes.

Docker Compose is the right choice for a small team when: the application fits on a single host, there are no hard requirements for HA or zero-downtime deployments, the team has limited Kubernetes expertise, and development speed is more important than operational resilience.

Kubernetes is justified when: the application needs to scale beyond a single host, multi-replica HA is required, or the team already has Kubernetes skills. Moving to Kubernetes prematurely imposes significant operational overhead. Start with Compose; migrate when you hit its limits.

Q106: Analyze vendor lock-in across Kubernetes and OpenShift.

Kubernetes is CNCF-governed open source. Workloads using only standard Kubernetes APIs are portable across AWS EKS, Google GKE, Azure AKS, on-premise, and any other conformant distribution. The ecosystem (Helm, Argo, Prometheus) is also open source.

OpenShift is Red Hat's commercial distribution. Workloads that use Routes, BuildConfigs, ImageStreams, or OperatorHub-deployed operators are tied to OpenShift. Migrating away requires replacing these with standard alternatives. Red Hat's support contract is valuable but creates dependency on their release cadence and pricing.

Q107: Defend a recommendation of OpenShift over vanilla Kubernetes for an enterprise.

For an enterprise that wants an integrated, vendor-supported platform:

- OpenShift bundles all required components: operator lifecycle management, image registry, monitoring (Prometheus/Grafana), logging (Elasticsearch/Loki), and CI/CD (Tekton/ArgoCD) — avoiding the need to assemble and maintain a custom stack.
- Red Hat provides enterprise support, security advisories, and a predictable upgrade path.
- Built-in multi-tenancy, audit logging, and SCCs reduce the effort needed to meet compliance requirements.
- The integrated developer console reduces time-to-deployment for application teams.

The trade-off is higher licensing costs and reduced flexibility compared to a self-managed Kubernetes stack.

Q108: Compare storage provisioning between Kubernetes and regular VM workloads.

VM workloads typically use storage attached directly to the VM (local disks, SAN/NFS mounts configured by system administrators). Changes require manual intervention and are not self-service.

Kubernetes abstracts storage through PVCs and StorageClasses. Developers request storage declaratively; the CSI driver provisions it automatically from the configured backend. This enables self-service, GitOps-driven storage management and integrates naturally with workload scaling.

Q109: A startup with two engineers must ship a containerized product quickly and cheaply. Recommend a container solution.

Recommendation: a managed Kubernetes service such as Google GKE Autopilot, AWS EKS Fargate, or DigitalOcean Kubernetes.

Justification: managed services eliminate the operational overhead of managing control plane nodes, upgrades, and etcd backups. Autopilot/Fargate remove even the need to manage worker nodes. A two-engineer team cannot afford to spend time on infrastructure when their competitive advantage is shipping product. Monthly cost for a small managed cluster starts at ~\$70–100, which is reasonable.

Alternative: if budget is extremely tight, start with Docker Compose on a single VM and migrate when growth demands it.

Q110: Compare Docker and Podman in terms of architecture and rootless security.

Docker uses a client-server architecture: the Docker CLI communicates with a daemon (dockerd) running as root. All container operations require root privileges, creating a large attack surface. A compromised daemon has root access to the host.

Podman is daemonless: each container is a direct child process of the user who started it. Podman supports rootless mode, running containers entirely as an unprivileged user. This significantly reduces the attack surface: a container escape gives an attacker only user-level privileges, not root.

A security-conscious team prefers Podman because it enforces the principle of least privilege by default.

Q111: Compare day-2 operational overhead of self-managed Kubernetes versus a managed service.

Self-managed Kubernetes (kubeadm, kops): the team is responsible for cluster upgrades (coordination of control plane + worker nodes), etcd backups, certificate rotation, cloud provider integration, monitoring the control plane, and incident response for any control plane failures.

Managed services (EKS, GKE, AKS): the provider manages the control plane, upgrades, and HA. The team only manages worker node pools and workloads. Day-2 overhead is reduced by 60–80%. The trade-off is reduced control and potentially higher cost.

Q112: A media-streaming company expects spiky, unpredictable global traffic. Recommend a containerized solution.

Recommendation: a managed Kubernetes service (e.g. GKE with Autopilot or AWS EKS with Karpenter node autoscaling) combined with Horizontal Pod Autoscaler (HPA) scaling on custom metrics (e.g. active video sessions, queue depth).

Justification: Kubernetes's HPA can scale pods within minutes in response to traffic spikes. Cluster Autoscaler or Karpenter provisions new nodes automatically. A CDN (CloudFront, Fastly) handles global edge caching to reduce origin load. For truly global multi-region traffic, a multi-region Kubernetes setup with global load balancing provides the lowest latency.

Q113: Defend whether a company that has outgrown Docker Swarm should migrate to Kubernetes.

Yes, migration is justified when:

- The application requires features Swarm lacks: advanced scheduling (affinity, taints/tolerations), custom resource definitions, sophisticated autoscaling, or a rich ecosystem of operators.
- The team has grown to the point where the operational investment in Kubernetes is spread across enough engineers.
- The long-term benefit of the Kubernetes ecosystem (Helm, Argo, Prometheus, Istio) outweighs the migration effort.

The migration effort is moderate: Docker Compose/Stack files must be converted to Kubernetes manifests (tools like Kompose help). Plan for a parallel-run period to validate behavior before cutting over.

Q114: Would you choose Kubernetes for a microservices architecture? Elaborate.

Yes. Kubernetes is an excellent fit for microservices:

- **Orchestration:** each microservice is a Deployment with its own replica count, resource requests, and rollout strategy.
- **Resilience:** liveness and readiness probes ensure unhealthy instances are automatically replaced and removed from traffic rotation.
- **Service discovery:** built-in DNS and ClusterIP Services make inter-service communication straightforward without external service registries.
- **Ecosystem:** Istio/Linkerd provide service mesh capabilities (mTLS, circuit breaking, distributed tracing) that are critical for large microservice architectures.

The operational overhead is justified by the resilience and scalability gains.

Q115: Would you choose Kubernetes for enterprise-grade applications? Elaborate.

Yes. Kubernetes has become the de facto enterprise platform for containerized workloads:

- **Scalability:** HPA, VPA, and cluster autoscaling handle workloads from small teams to thousands of nodes.
- **Community:** the CNCF ecosystem is the largest in cloud-native infrastructure, ensuring long-term support and a large pool of skilled engineers.
- **Feature richness:** RBAC, NetworkPolicies, Pod Security Admission, audit logging, and the Operator pattern cover enterprise requirements for security, compliance, and lifecycle management.
- **Cloud portability:** CNCF conformance ensures workloads are not locked to a single cloud vendor.

Q116: Would you choose OpenShift for a microservices architecture? Elaborate.

Yes, particularly in enterprise environments:

- **Orchestration:** OpenShift uses the same Kubernetes primitives (Deployments, Services) enhanced with Routes for easier HTTP exposure and built-in TLS.
- **Resilience:** the same probe-based health management as Kubernetes, enhanced by OpenShift's OperatorHub, which provides production-grade operators for databases, messaging systems, and other stateful services used in microservices architectures.
- **Developer experience:** S2I and the developer console reduce the time from code commit to running microservice, which is valuable when managing many independent services.

The main drawback is that OpenShift-specific APIs (Routes, ImageStreams) reduce portability compared to plain Kubernetes.

Q117: Would you choose OpenShift for enterprise-grade applications? Elaborate.

Yes, OpenShift is an especially strong choice for enterprise-grade applications:

- **Scalability:** built on Kubernetes, it inherits all its scaling capabilities, with the addition of the MachineConfigOperator for managing node configuration at scale.
- **Community and support:** Red Hat provides enterprise SLAs, security patches, and a certified operator catalog (OperatorHub). This reduces the risk of running critical workloads on community-supported components.

- Feature richness: integrated monitoring, logging, CI/CD, image registry, and network policies are available out of the box, reducing the number of third-party tools the operations team must manage.
- Compliance: built-in SCCs, audit logging, FIPS compliance options, and regular CVE patching make OpenShift well-suited for regulated industries such as finance and healthcare.